

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Programa de licenciatura en Ingeniería en Computadores

Curso:
CE-1101 Introducción a la programación
Programación Orientada a Objetos en Python:
Tienda de videojuegos

Grupo: 5
Amanda Villalobos Benavides
Luis Diego Murillo Matarrita
Sebastián Solano Porras

Descripción del problema

Jomvet (จอมเวท) es una empresa de videojuegos en línea tailandesa encargada de vender e instalar videojuegos de programadores y compañías amateur en el mercado del sudeste asiático. Su visión es ser una plataforma ampliamente conocida y eficaz como Steam. En este momento Jomvet gestiona sus compras e interacciones con el cliente de forma manual y por medio de llamadas, lo cuál es ineficiente para ellos y nada práctico para los usuarios. Desean crear una plataforma en donde cada videojuego esté clasificado de acuerdo a su género: indie, acción, RPG, deportes y novelas visuales. Algunos de sus juegos son:

- **Six nights at Bon's Burgers:**
 - Categoría: Indie
 - Precio: \$ 6.15
 - Descuento: Sin descuento
- **Better call Saul: Ace Attorney special:**
 - Categoría: Novela Visual
 - Precio: \$20
 - Descuento: 30%
- **Haaland sports island x Mii sports island crossover:**
 - Categoría: Deportes
 - Precio: \$45
 - Descuento: 5%
- **Joker Arkham series full pack:**
 - Categoría: Acción
 - Precio: \$160
 - Descuento: 30%
- **First Fantasy VIII: Sephiroth's backstory**
 - Categoría: RPG
 - Precio: \$300
 - Descuento: Sin descuento
- **Labubu Escape**
 - Categoría: Acción
 - Precio: \$10
 - Descuento: Sin descuento
- **Legend of Link: Majora's Ocarina**
 - Categoría: RPG
 - Precio: \$180
 - Descuento: 15%
- **Giant nightmares: The final nap**
 - Categoría: Indie
 - Precio: \$50

- Descuento: 20%
- **PokeDigimon: Scarlet&Violet**
 - Categoría: Novela Visual
 - Precio: \$200
 - Descuento: Sin descuento

Además de su clasificación, se busca que la plataforma guarde los videojuegos de interés para el usuario en un “carrito de compras” para animar al usuario a realizar una compra. Este carrito suma los precios de los videojuegos seleccionados con el descuento aplicado de una vez. El carrito no permite al usuario guardar el mismo videojuego dos veces. Ya realizada una compra la plataforma genera un recibo indicando el total pagado y la fecha de transacción, el usuario no puede realizar una compra si su total por pagar es 0 o negativo.

¿Por qué es un problema relacionado con ingeniería en computadores?

Las ingenierías tienen como propósito principal ingeniar soluciones u optimizar procesos utilizando la tecnología. La ingeniería en computadores propone soluciones en informática y máquinas digitales. En este caso se presenta una empresa en busca de una creación de software para facilitar la interacción entre producto y cliente. Programación orientada a objetos se utiliza para asignar cada videojuego a una categoría y posteriormente relacionarlos a un carrito y una función de compra, que solucionaría el problema de la compañía ficticia Jomvet (จอมเวท), problema que muchas empresas en fase de iniciación enfrentan en la realidad y podría solucionar un ingeniero en computadores.

Soluciones

POO en descuentos y modificación de precios:

En toda tienda, es bien sabido que existen posibles descuentos aplicables en cada producto de la misma. En una tienda física, para representar esto, los empleados ponen en el estante de productos que tienen dicho descuento una etiqueta con el descuento a aplicar, en lugar de ponerlo a cada producto por separado.

En la tienda virtual del ejemplo, es parecido, en lugar de escribir líneas de código que represente dicho descuento que vuelven el producto más complicado de trabajar, se puede crear una clase “Descuento”, la cual existe sin necesidad de que el objeto videojuego/ programa exista. Dicha clase trabajaría solamente con los objetos VideoJuegos y tendría los atributos: nombre, porcentaje y tiempoAplicable. Al mismo tiempo, los métodos de la clase “Descuento” son: setNombre(), setPorcentaje(), setTimepoAplicable, getNombre(), getPorcentaje() y

getTiempoAplicable. Estos parámetros son indispensables ya que de otro modo no se podrían modificar o acceder a sus atributos, aplicando Encapsulamiento.

Con esto, surge la duda: ¿puede un parámetro de la clase Descuento modificar un atributo de la clase Videojuego? No es lo ideal. Para esto, la clase VideoJuego debería poseer un método que le permita obtener al atributo “porcentaje” de la clase Descuento, y con este, verifique si es aplicable y modifique su propio atributo de precio.

Un ejemplo práctico para entender esta idea, son los descuentos de primavera. Se crea la clase con sus respectivos atributos, los cuales a su vez pueden ser modificados a través de los métodos que posee. Con esto, la clase VideoJuego, a través de un método propio, accede a los atributos de la clase Descuento, verifica si es posible la aplicación, y modifica a sí misma su atributo de precio.

POO para el carrito de compras y transacciones: La clase del carrito de compras tendría un conjunto de POO específico para asegurarse de que no se repitan los videojuegos a comprar. La clase trabajaría con los objetos Videojuegos y trabajaría con la clase Carrito que tendría métodos: agregarJuego(), calcularTotal() y confirmarTransacción() que conecten con un objeto Recibo. Se tiene la clase Videojuego con atributos (self, nombre, categoría, precio, descuento). Su método principal es ObtenerPrecio (precio base - porcentaje de descuento). La clase carrito tendría los atributos (self, IDjugador, listaJuegos). ListaJuegos no se trabajaría como una lista sino como un SET, esto es crucial ya que por definición esta estructura de datos no permite elementos duplicados. El método calcularTotal invoca el ObtenerPrecio de cada juego y luego suma todos obteniendo el total. El método confirmarTransacción da pie al recibo. Finalmente, la clase Recibo (self, IDTransaccion, Fecha, listaItems, montoTotal). Estos dos últimos simplemente imprimen los resultados de las operaciones del carrito.

POO para gestión de información y seguridad de la plataforma: Un programador o empresa no puede modificar los precios ni descuentos de sus videojuegos, solamente dirigentes seleccionados de Jomvet pueden hacerlo, ellos solo pueden ver las estadísticas de sus propios juegos. Para establecer la seguridad del sistema se utilizan los conceptos de Herencia y Polimorfismo. La clase base sería Usuario con atributos (self, ID, nombre, correo, contraseña) y su método Información simplemente es un print que la expone. Habrían subclases que son DesarrolladorAmateur, Dirigente y Cliente. Todos tienen los mismos atributos de su clase madre (Usuario) y propios. Los atributos propios para cliente: billeteraJomvetcoins, historialCompras y carritoCompras. Sus métodos propios son agregarCarrito() y pagar(). Luego está el DesarrolladorAmateur cuyos atributos propios son: estudio, juegosPublicados, cuentaBancariaAsociada. Sus métodos además de Información son: subirJuego(), modificarJuego() y verEstadísticas(). Por último la subclase Dirigente tiene como atributo propio: seguridadJomvet. Sus métodos son: aprobarVideojuego(), banearUsuario(), modificarPrecio(), verEstadísticasJomvet(), administrarTransaccionesGlobales().